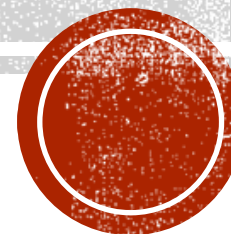


COUR REACT JS



INTRODUCTION À REACT ET SON ÉCOSYSTÈME

- React est une bibliothèque JavaScript open source développée par Facebook pour construire des interfaces utilisateur interactives et dynamiques. Il est devenu l'une des bibliothèques les plus populaires pour le développement d'applications web et mobiles. React se concentre sur la création de composants réutilisables, ce qui le rend idéal pour les applications à grande échelle.



INSTALLATION DE L'ENVIRONNEMENT DE DÉVELOPPEMENT

- Pour commencer à travailler avec React, vous devez mettre en place un environnement de développement adéquat. React peut être configuré manuellement à l'aide de Webpack, Babel et d'autres outils, mais la méthode la plus simple pour les débutants est d'utiliser **Create React App (CRA)**, un outil officiel de Facebook pour créer des projets React préconfigurés.

- **Installation avec Create React App (CRA) :**

1. Assurez-vous d'avoir Node.js installé sur votre système. Si ce n'est pas le cas, téléchargez et installez Node.js depuis [le site officiel](#).
2. Ouvrez votre terminal et installez Create React App en utilisant npm (Node Package Manager) :

```
npm install -g create-react-app
```

3. Une fois l'installation terminée, vous pouvez créer un nouveau projet React en utilisant la commande suivante :

```
npx create-react-app mon-projet-react
```

Remplacez mon-projet-react par le nom de votre choix.

4. Naviguez dans le répertoire de votre projet :

```
cd mon-projet-react
```

```
npm start
```

Démarrez l'application de développement :



STRUCTURE DES DOSSIERS

- Après création, votre projet devrait ressembler à ceci :
- pour que le projet soit construit, ces fichiers doivent exister avec des noms de fichiers exacts :
- public/index.html est le modèle de page ;
- src/index.js est le point d'entrée JavaScript.
- Vous pouvez supprimer ou renommer les autres fichiers.
- Vous pouvez créer des sous-répertoires à l'intérieur de src. Pour des reconstructions plus rapides, seuls les fichiers à l'intérieur src sont traités par webpack. Vous devez mettre tous les fichiers JS et CSS à l'intérieur src , sinon Webpack ne les verra pas.
- Seuls les fichiers à l'intérieur public peuvent être utilisés à partir de public/index.html. Lisez les instructions ci-dessous pour utiliser les ressources JavaScript et HTML.
- Vous pouvez toute fois créer davantage de répertoires de niveau supérieur. Ils ne seront pas inclus dans la version de production, vous pourrez donc les utiliser pour des choses comme la documentation.

```
my-app/  
  README.md  
  node_modules/  
  package.json  
  public/  
    index.html  
    favicon.ico  
  src/  
    App.css  
    App.js  
    App.test.js  
    index.css  
    index.js  
    logo.svg
```



COMPOSANT CLASS ET COMPOSANT FONCTION

- En React, il existe deux principales façons de créer des composants : les composants de classe (Class Components) et les composants fonctionnels (Functional Components). Les deux ont des cas d'utilisation différents et des avantages spécifiques.
- Composant de classe (Class Component) :
- Les composants de classe sont des classes JavaScript qui étendent la classe `React.Component`.
- Ils disposent d'un cycle de vie complet et permettent de gérer l'état interne.
- Un exemple de composant de classe :

```
jsx Copy code  
  
import React, { Component } from 'react';  
  
class MonComposantClass extends Component {  
  render() {  
    return <div>Je suis un composant de classe.</div>;  
  }  
}
```



- Composant fonctionnel (Functional Component) :
- Les composants fonctionnels sont des fonctions JavaScript qui prennent des propriétés en entrée et renvoient du JSX.
- Ils sont plus simples à écrire et plus légers en termes de syntaxe.
- Les composants fonctionnels sont recommandés lorsque vous n'avez pas besoin de gérer l'état interne.
- Un exemple de composant fonctionnel :

jsx

Copy code

```
import React from 'react';

function MonComposantFonctionnel() {
  return <div>Je suis un composant fonctionnel.</div>;
}
```

Avec l'évolution de React, l'utilisation de composants fonctionnels est devenue la norme en raison de leur simplicité et de la possibilité d'utiliser des fonctionnalités avancées telles que les hooks pour gérer l'état et les effets.



CRÉATION D'UN COMPOSANT

- Sur le Web, HTML nous permet de créer des documents riches et structurés avec son ensemble de balises intégrées comme `<h1>` et `<li>`:
- Ce balisage représente cet article `<article>`, son titre `<h1>` et une table des matières (abrégée) sous forme de liste ordonnée `<ol>`. Un balisage comme celui-ci, combiné avec CSS pour le style et JavaScript pour l'interactivité, se trouve derrière chaque barre latérale, avatar, modal, liste déroulante – chaque élément de l'interface utilisateur que vous voyez sur le Web.
- React vous permet de combiner votre balisage, CSS et JavaScript dans des « composants » personnalisés, des éléments d'interface utilisateur réutilisables pour votre application. Le code de la table des matières que vous avez vu ci-dessus pourrait être transformé en un `<TableOfContents />` composant que vous pourriez afficher sur chaque page. Sous le capot, il utilise toujours les mêmes balises HTML comme `<article>`, `<h1>`, etc.

```
<article>
  <h1>My First Component</h1>
  <ol>
    <li>Components: UI Building Blocks</li>
    <li>Defining a Component</li>
    <li>Using a Component</li>
  </ol>
</article>
```



- Tout comme avec les balises HTML, vous pouvez composer, ordonner et imbriquer des composants pour concevoir des pages entières. Par exemple, la page de documentation que vous lisez est composée de composants React :

Au fur et à mesure que votre projet grandit, vous remarquerez que bon nombre de vos conceptions peuvent être composées en réutilisant des composants que vous avez déjà écrits, accélérant ainsi votre développement. Notre table des matières ci-dessus pourrait être ajoutée à n'importe quel écran avec `<TableOfContents />` !

```
<PageLayout>
  <NavigationHeader>
    <SearchBar />
    <Link to="/docs">Docs</Link>
  </NavigationHeader>
  <Sidebar />
  <PageContent>
    <TableOfContents />
    <DocumentationText />
  </PageContent>
</PageLayout>
```



- Étape 1 : Exporter le composant
- Le export default est une syntaxe JavaScript standard (non spécifique à React). Il vous permet de marquer la fonction principale dans un fichier afin de pouvoir l'importer ultérieurement à partir d'autres fichiers. (En savoir plus sur l'importation et exportation de composants !)

Étape 1 : Définir la fonction

Avec fonction MonComposant() { } vous définissez une fonction JavaScript avec le nom Profile.

Étape 2 : Ajouter un balisage

Le composant renvoie une <div /> balise avec les. <div /> est écrit comme du HTML, mais c'est en fait du JavaScript sous le capot ! Cette syntaxe s'appelle JSX et vous permet d'intégrer du balisage dans JavaScript.

Les instructions return peuvent être écrites sur une seule ligne, comme dans ce composant :

```
return ;
```

Mais si votre balisage ne se trouve pas entièrement sur la même ligne que le return mot-clé, vous devez le mettre entre parenthèses :

```
return (  
  <div>  
      
  </div>  
);
```

```
jsx Copy code  
  
import React from 'react';  
  
function MonComposant() {  
  return (  
    <div>  
      <h1>Mon Composant Fonctionnel</h1>  
      <p>Ce composant est créé en tant que fonction.</p>  
    </div>  
  );  
}  
  
export default MonComposant;
```

Étape 3 : Exporter le composant

Le export default est une syntaxe JavaScript standard (non spécifique à React). Il vous permet de marquer la fonction principale dans un fichier afin de pouvoir l'importer ultérieurement à partir d'autres fichiers. (En savoir plus sur l'importation et exportation de composants !)



IMPORTATION ET EXPORTATION DE COMPOSANTS

- React utilise un système d'importation et d'exportation pour organiser et réutiliser des composants. Cette leçon explique comment importer des composants d'autres fichiers et comment les exporter pour les utiliser ailleurs dans votre application.

Vous pouvez exporter un composant en tant qu'exportation par défaut, ce qui signifie qu'il peut être importé sous n'importe quel nom dans d'autres fichiers.

- Exemple de composant exporté par défaut :

Vous pouvez exporter un composant en tant qu'exportation par défaut, ce qui signifie qu'il peut être importé sous n'importe quel nom dans d'autres fichiers.

Exemple de composant exporté par défaut :

Pour utiliser un composant exporté dans un autre fichier, vous utilisez l'instruction import. Vous pouvez importer des composants exportés par défaut ou de manière nommée.

Exemple d'importation d'un composant exporté par défaut :

Important : n'oubliez pas que le composant parent c'est App

```
jsx
import React from 'react';

function MonComposant() {
  return <p>Je suis un composant exporté par défaut.</p>;
}

export default MonComposant;
```

```
jsx
import React from 'react';
import MonComposant from './MonComposant';

function App() {
  return (
    <div>
      <h1>Mon Application React</h1>
      <MonComposant />
    </div>
  );
}

export default App;
```



JSX

- JSX (JavaScript XML) est une extension de syntaxe utilisée par React pour décrire l'apparence des éléments de l'interface utilisateur. C'est une syntaxe qui ressemble à du HTML, mais elle est en réalité du JavaScript. JSX est une partie essentielle de React, car il permet de définir la structure de votre interface utilisateur de manière déclarative.
- En React, vous créez des éléments JSX pour représenter les éléments de votre interface utilisateur. Voici comment vous pouvez créer un élément JSX simple :

```
jsx Copy code  
  
const elementJSX = <h1>Bienvenue dans mon application React</h1>;
```

- Dans cet exemple, elementJSX est une variable qui contient un élément JSX représentant un titre <h1>.
- Utilisation d'expressions dans JSX :
- Vous pouvez intégrer des expressions JavaScript dans du JSX en les entourant de {}. Cela vous permet d'injecter des valeurs dynamiques dans vos éléments JSX. Par exemple :

```
jsx Copy code  
  
const nom = "John";  
const elementJSX = <p>Bonjour, {nom} !</p>;
```

- Dans cet exemple, la valeur de la variable nom est insérée dans l'élément JSX, ce qui affiche "Bonjour, John !" à l'utilisateur.
- Vous pouvez également spécifier des attributs HTML et des valeurs pour ces attributs dans vos éléments JSX, de la même manière que vous le feriez en HTML classique



- En React, vous utiliserez souvent JSX pour définir la structure de vos composants. Voici un exemple de composant fonctionnel utilisant JSX :
- Le composant MonComposant renvoie un élément JSX qui définit la structure du composant. Lorsque ce composant est rendu, le JSX est transformé en HTML et affiché à l'utilisateur.
- Règles JSX importantes :
 - Les éléments JSX doivent avoir une seule racine parente. Vous ne pouvez pas retourner plusieurs éléments de niveau supérieur directement. Pour cela, vous pouvez envelopper les éléments dans une div ou utiliser des fragments.
 - Les noms des éléments JSX doivent commencer par une lettre majuscule pour les composants personnalisés. Les éléments HTML en minuscules sont interprétés comme des balises HTML standard.
 - Le code JavaScript dans les {} est interprété comme une expression, vous ne pouvez pas y inclure des déclarations ou des instructions.

```
jsx Copy code  
  
import React from 'react';  
  
function MonComposant() {  
  return (  
    <div>  
      <h1>Mon Composant React</h1>  
      <p>Ceci est un exemple de composant React utilisant JSX.</p>  
    </div>  
  );  
}
```

```
App.js Télécharger Réinitialiser Fourchette  
  
1 export default function Avatar() {  
2   const avatar = 'https://i.imgur.com/7vQD0fPs.jpg';  
3   const description = 'Gregorio Y. Zara';  
4   return (  
5     <img  
6       className="avatar"  
7       src={avatar}  
8       alt={description}  
9     />  
10  );  
11 }  
12
```



LES PROPS

- Les props (abréviation de "propriétés") sont des mécanismes essentiels en React pour transmettre des données d'un composant parent à un composant enfant. Les props permettent de passer des informations dynamiques à un composant, ce qui rend les composants réutilisables et configurables.
- Exemple avec des props :
- Prenons l'exemple d'un composant "Message" qui affiche un message avec un titre et un contenu. Ce composant prendra deux props, "title" et "content", pour personnaliser le message.

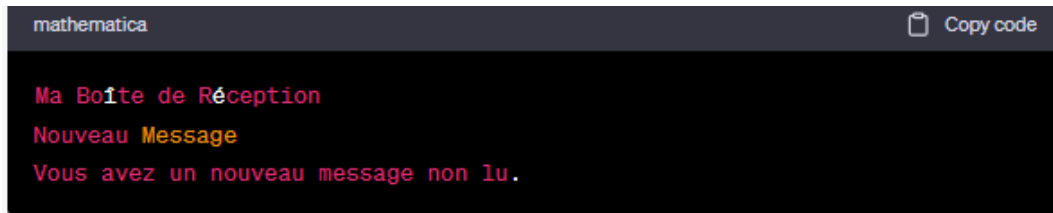
Dans le composant "Message", nous utilisons les props pour afficher le titre et le contenu. Les props sont accessibles à l'intérieur de la fonction Message en tant qu'objet JavaScript. Vous pouvez accéder aux valeurs des props en utilisant `props.nomDeLaPropriété`.

```
jsx Copy code  
  
import React from 'react';  
  
function Message(props) {  
  return (  
    <div className="message">  
      <h2>{props.title}</h2>  
      <p>{props.content}</p>  
    </div>  
  );  
}  
  
export default Message;
```



- Utilisation du composant "Message" avec des props :
- Maintenant, dans un autre composant, par exemple "App", nous pouvons utiliser le composant "Message" en lui passant des valeurs spécifiques pour les props. Cela nous permet de personnaliser le contenu du message.
- Dans le composant "App", nous utilisons le composant "Message" en lui passant les props "title" et "content". Ces valeurs personnalisées seront utilisées par le composant "Message" pour afficher un message avec le titre "Nouveau Message" et le contenu "Vous avez un nouveau message non lu."
- Ainsi, les props permettent de personnaliser et de configurer les composants de manière dynamique, ce qui les rend réutilisables dans différentes parties de l'application. Les props sont essentielles pour la communication entre les composants en React.

Voici comment cela s'afficherait à l'écran :



```
import React from 'react';
import Message from './Message';

function App() {
  return (
    <div>
      <h1>Ma Boîte de Réception</h1>
      <Message
        title="Nouveau Message"
        content="Vous avez un nouveau message non lu."
      />
    </div>
  );
}

export default App;
```



PROPS.CHILDREN

- `props.children` est une propriété spéciale en React qui permet de transmettre et de manipuler des éléments enfants (composants ou contenu) à l'intérieur d'un composant parent. Cette fonctionnalité est utile lorsque vous souhaitez encapsuler du contenu dans un composant réutilisable.
- Utilisation de `props.children` :
- Imaginons que vous souhaitiez créer un composant "Panneau" qui peut afficher un titre et un contenu. Vous pouvez utiliser `props.children` pour injecter du contenu arbitraire dans ce composant.
- Composant "Panneau" avec `props.children` :

Dans ce composant "Panneau", nous utilisons `props.children` pour afficher le contenu que nous passerons en utilisant ce composant. `props.titre` est une autre propriété qui permet de spécifier un titre pour le panneau.

```
jsx Copy code  
  
import React from 'react';  
  
function Panneau(props) {  
  return (  
    <div className="panneau">  
      <h2>{props.titre}</h2>  
      <div className="contenu">  
        {props.children}  
      </div>  
    </div>  
  );  
}  
  
export default Panneau;
```



- Maintenant, vous pouvez utiliser le composant "Panneau" en passant du contenu à l'intérieur de celui-ci. Par exemple :
- Dans l'exemple ci-dessus, le composant "Panneau" est utilisé avec du contenu enfant. Le contenu entre les balises <Panneau> est transmis au composant "Panneau" en utilisant props.children. Cela permet d'encapsuler différents types de contenu, y compris des composants React, à l'intérieur du "Panneau" de manière dynamique.
- Utiliser props.children permet de créer des composants React plus flexibles et réutilisables qui peuvent être configurés avec divers contenus. C'est une technique puissante pour construire des interfaces utilisateur modulaires.

```
jsx

import React from 'react';
import Panneau from './Panneau';

function App() {
  return (
    <div>
      <h1>Mon Application React</h1>
      <Panneau titre="Mon Panneau">
        <p>Contenu du panneau.</p>
        <button>Cliquez-moi</button>
      </Panneau>
    </div>
  );
}

export default App;
```

```
bash

Mon Application React
Mon Panneau
Contenu du panneau.
Cliquez-moi
```



CYCLE DE VIE

- Le cycle de vie d'un composant est un concept important en React. Il fait référence aux différentes étapes par lesquelles un composant passe, de sa création à sa destruction. Comprendre le cycle de vie d'un composant est essentiel pour effectuer des opérations telles que l'initialisation, la mise à jour et la suppression de données.
- Cependant, il est important de noter que depuis React 16.3, de nombreux aspects du cycle de vie des composants ont été dépréciés au profit de l'utilisation des hooks, comme `useEffect`, dans les composants fonctionnels. Les composants de classe continuent d'utiliser les méthodes de cycle de vie, mais elles sont progressivement remplacées par les hooks.
- Voici un aperçu simplifié des principales étapes du cycle de vie d'un composant de classe en React :
- Montage (Mounting) :
 - `constructor()` : Le constructeur est appelé lorsque le composant est initialisé. C'est l'endroit où vous pouvez initialiser l'état initial du composant.
 - `render()` : La méthode `render()` est responsable du rendu du composant. Elle retourne un élément JSX qui représente la structure de l'interface utilisateur.
- Mise à jour (Updating) :
 - `componentDidMount()` : Cette méthode est appelée immédiatement après le rendu initial du composant.
 - `shouldComponentUpdate(nextProps, nextState)` : Elle détermine si le composant doit être mis à jour en fonction des nouvelles propriétés (props) et de l'état (state). Elle renvoie `true` ou `false`.
 - `componentDidUpdate(prevProps, prevState)` : Elle est appelée après la mise à jour du composant et permet d'effectuer des opérations supplémentaires.
- Démontage (Unmounting) :
 - `componentWillUnmount()` : Cette méthode est appelée juste avant que le composant soit retiré de l'interface utilisateur.



- Ces méthodes de cycle de vie étaient couramment utilisées dans les composants de classe pour effectuer des opérations telles que l'initialisation de données, les requêtes réseau, la gestion des mises à jour de l'interface utilisateur, etc.
- Cependant, avec l'introduction des hooks dans les composants fonctionnels, le cycle de vie devient moins pertinent. Par exemple, `useEffect` permet d'effectuer des opérations similaires à celles des méthodes du cycle de vie, mais de manière plus modulaire.
- Il est recommandé d'utiliser les composants fonctionnels et les hooks pour les nouvelles applications React, car ils offrent une syntaxe plus légère et simplifient la gestion de l'état et des effets.
- Les Hook sont entamé les prochaine lecon



RENDU CONDITIONNELS

- En React, les rendus conditionnels sont courants pour afficher du contenu en fonction d'une condition donnée. Vous pouvez réaliser des rendus conditionnels de deux manières principales : en utilisant des instructions if dans le corps de la fonction de rendu ou en utilisant l'opérateur ternaire.
- Rendu Conditionnel avec "if" :
- Imaginons que vous ayez une variable estConnecte qui détermine si un utilisateur est connecté ou non. Vous pouvez utiliser une instruction if pour afficher du contenu en conséquence :

Dans cet exemple, si estConnecte est true, le composant affiche "Bienvenue, utilisateur connecté !". Sinon, il affiche "Veuillez vous connecter pour accéder."

```
jsx

import React from 'react';

function MonComposant() {
  const estConnecte = true;

  if (estConnecte) {
    return <p>Bienvenue, utilisateur connecté !</p>;
  } else {
    return <p>Veuillez vous connecter pour accéder.</p>;
  }
}

export default MonComposant;
```



- Rendu Conditionnel avec l'Opérateur Ternaire :
- Vous pouvez également utiliser l'opérateur ternaire (condition ? vrai : faux) pour effectuer des rendus conditionnels de manière concise :

```
import React from 'react';
function MonComposant() {
  const estConnecte = true;
  return (
    <div>
      {
        estConnecte ? <p>Bienvenue, utilisateur connecté !</p>
          : <p>Veuillez vous connecter pour accéder.</p>
      }
    </div>
  );
}
export default MonComposant;
```



RAPPEL LA MÉTHODE .MAP() EN JAVASCRIPT

- La méthode `.map()` est une fonction de tableau en JavaScript qui permet de parcourir un tableau existant et de créer un nouveau tableau en appliquant une fonction à chaque élément du tableau d'origine. Elle est utilisée pour effectuer des transformations sur chaque élément d'un tableau, générant ainsi un nouveau tableau avec les résultats de ces transformations.

- La syntaxe de base de `.map()` est la suivante :

```
javascript Copy code  
  
const nouveauTableau = tableauSource.map(fonctionTransformation);
```

- `tableauSource` : Le tableau d'origine que vous souhaitez parcourir.
- `fonctionTransformation` : Une fonction qui sera exécutée pour chaque élément du tableau source. Cette fonction prend en général trois paramètres : l'élément actuel, l'index de l'élément et le tableau source lui-même. Elle renvoie la valeur transformée qui sera ajoutée au nouveau tableau.
- Exemple d'utilisation de `.map()` pour doubler chaque élément d'un tableau :

```
javascript  
  
const nombres = [1, 2, 3, 4, 5];  
const doubles = nombres.map(function(nombre) {  
    return nombre * 2;  
});  
  
// "doubles" contient maintenant [2, 4, 6, 8, 10]
```



CRÉATION DE LISTES EN REACT

- La création de listes est une tâche courante en développement d'applications React, car vous devez souvent afficher des ensembles de données sous forme de liste. Vous pouvez utiliser la méthode `.map()` pour générer dynamiquement des éléments à partir d'un tableau de données.
- Supposons que vous ayez un tableau de noms d'utilisateurs que vous souhaitez afficher dans une liste à puces. Voici comment vous pouvez le faire :

Dans cet exemple, nous avons un tableau `utilisateurs` contenant les noms des utilisateurs. Nous utilisons la méthode `.map()` pour itérer sur ce tableau et générer dynamiquement des éléments `<li>` pour chaque nom d'utilisateur. Nous utilisons la clé `key` pour aider React à identifier de manière unique chaque élément de la liste.

L'élément de liste `<ul>` contient les éléments générés, créant ainsi une liste à puces de noms d'utilisateurs.

Sortie attendue :

Lorsque vous utilisez le composant `ListeUtilisateurs`, la sortie attendue serait une liste à puces affichant les noms d'utilisateurs comme suit :

Alice
Bob
Charlie
David

```
jsx Copy code

import React from 'react';

function ListeUtilisateurs() {
  const utilisateurs = ['Alice', 'Bob', 'Charlie', 'David'];

  const elementsListe = utilisateurs.map((utilisateur, index) => (
    <li key={index}>{utilisateur}</li>
  ));

  return (
    <ul>
      {elementsListe}
    </ul>
  );
}

export default ListeUtilisateurs;
```

UTILISATION D'IMAGES AVEC REACT

- Ajout d'Images à votre Projet :
- Tout d'abord, assurez-vous que vos images sont présentes dans le répertoire de votre projet. Vous pouvez les stocker dans un dossier spécifique, par exemple, dans le répertoire "public" de votre application React.
- Importation de l'Image :
- Dans le composant où vous souhaitez utiliser l'image, importez-la en utilisant la syntaxe import de JavaScript. Par exemple :

```
jsx  
  
import monImage from './images/monImage.jpg';
```

- Dans cet exemple, nous avons importé une image appelée "monImage.jpg" située dans le dossier "images" du répertoire racine de l'application.
- Utilisation de l'Image dans le Composant :
- Une fois l'image importée, vous pouvez l'afficher en utilisant la balise

dans le JSX de votre composant. Par exemple :

Assurez-vous d'utiliser la variable monImage comme valeur de l'attribut src de la balise . Vous pouvez également spécifier un texte de remplacement dans l'attribut alt pour décrire l'image.

Mise en Ligne :

Lorsque vous déployez votre application React, les images seront incluses dans le bundle de production. Assurez-vous que le chemin d'importation des images est correctement configuré pour fonctionner en production.

```
jsx  
  
import React from 'react';  
import monImage from './images/monImage.jpg';  
  
function MonComposant() {  
  return (  
    <div>  
      <h1>Mon Composant avec une Image</h1>  
      <img src={monImage} alt="Description de l'image" />  
    </div>  
  );  
}  
  
export default MonComposant;
```

CSS EN REACT

- Styles CSS internes :
- Vous pouvez également définir des styles CSS directement dans le fichier JavaScript/JSX de votre composant en utilisant des objets JavaScript pour les styles.
- Exemple :

```
jsx Copy code

import React from 'react';

const styles = {
  titre: {
    fontSize: '24px',
    fontWeight: 'bold',
  },
  paragraphe: {
    fontSize: '16px',
    color: 'blue',
  },
};

function MonComposant() {
  return (
    <div>
      <h1 style={styles.titre}>Mon Composant React</h1>
      <p style={styles.paragraphe}>Ceci est un exemple de texte.</p>
    </div>
  );
}

export default MonComposant;
```



- **Styles CSS en ligne :**

- Vous pouvez également définir des styles en ligne directement dans les éléments JSX.

- Exemple :

```
jsx Copy code  
  
import React from 'react';  
  
function MonComposant() {  
  const styleTitre = {  
    fontSize: '24px',  
    fontWeight: 'bold',  
  };  
  
  return (  
    <div>  
      <h1 style={styleTitre}>Mon Composant React</h1>  
      <p style={{ fontSize: '16px', color: 'blue' }}>Ceci est un exemple de  
    </div>  
  );  
}  
  
export default MonComposant;
```



- Importez le fichier CSS dans le composant React :
- Dans le fichier JSX de votre composant React, vous pouvez importer le fichier CSS en utilisant l'instruction import. Voici comment vous pouvez faire cela :

```
jsx Copy code  
  
import React from 'react';  
import './styles.css'; // Importation du fichier CSS  
  
function MonComposant() {  
  return (  
    <div>  
      <h1>Mon Composant React</h1>  
      <p>Ceci est un exemple de texte avec des styles CSS.</p>  
    </div>  
  );  
}  
  
export default MonComposant;
```



LES EVENEMENTS

- **Déclaration d'un Gestionnaire d'Événements** : Pour déclarer un gestionnaire d'événements dans un composant React, vous ajoutez simplement un attribut d'événement à un élément JSX et lui associez une fonction. Par exemple, pour gérer un événement de clic :

```
jsx
<button onClick={maFonctionDeGestion}>Cliquez-moi</button>
```

- Définition de la Fonction de Gestion :
- Vous devez ensuite définir la fonction `maFonctionDeGestion` quelque part dans votre composant. Cette fonction sera exécutée lorsque l'événement se produit.

```
jsx
function maFonctionDeGestion() {
  // Logique de gestion de l'événement ici
}
```

- Passage d'Arguments :
- Vous pouvez également passer des arguments personnalisés à votre fonction de gestion en utilisant des fonctions fléchées ou `.bind()`. Par exemple, si vous souhaitez passer un ID lorsqu'un élément est cliqué :

```
<button onClick={() => maFonctionDeGestionAvecArguments(id)}>Cliquez-moi</button>
```



LES HOOK

- Les hooks sont des fonctions spéciales fournies par React pour ajouter des fonctionnalités à des composants fonctionnels. Les hooks permettent aux composants fonctionnels de gérer l'état, les effets, le contexte et d'autres aspects sans avoir besoin de convertir ces composants en composants de classe.
- Le principe fondamental des hooks est de permettre aux composants fonctionnels d'avoir un état interne, de s'abonner aux effets secondaires (comme les requêtes réseau ou les événements DOM) et d'interagir avec le cycle de vie du composant, tout en conservant la simplicité et la lisibilité inhérentes aux composants fonctionnels.
- Les hooks sont conçus pour être réutilisables et composables, ce qui signifie que vous pouvez les combiner pour créer des fonctionnalités plus complexes et les partager entre différents composants. Ils offrent une manière plus modulaire de gérer la logique métier dans vos composants React, facilitant la création de composants réactifs et évolutifs. Chaque hook a un but spécifique, ce qui permet aux développeurs de choisir les fonctionnalités à utiliser en fonction des besoins de leur application.



USESTATE

- Le hook `useState` est utilisé pour gérer l'état local dans un composant fonctionnel. Il permet au composant de mémoriser des valeurs qui peuvent être modifiées et affectent le rendu du composant. `useState` renvoie un tableau contenant deux éléments : la valeur de l'état actuel et une fonction pour la mettre à jour. Voici comment il fonctionne :
- Déclaration de l'état initial :
- Vous pouvez déclarer une variable d'état avec `useState` en lui passant une valeur initiale. Par exemple :

```
jsx Copy code  
  
const [compteur, setCompteur] = useState(0);
```

- Ici, `compteur` est l'état et `setCompteur` est la fonction pour le mettre à jour. L'état initial est défini à 0.
- Utilisation de l'état :
- Vous pouvez utiliser la variable d'état `compteur` dans votre composant pour afficher ou effectuer des opérations en fonction de sa valeur actuelle.

```
jsx Copy code  
  
<p>Compteur : {compteur}</p>
```



- Mise à jour de l'état :
- Pour mettre à jour l'état, vous utilisez la fonction `setCompteur`, par exemple, lorsqu'un bouton est cliqué.
- Appeler `setCompteur` avec une nouvelle valeur déclenchera la mise à jour du composant et le rendu avec la nouvelle valeur de l'état.

```
import React, { useState } from 'react';

function ComposantAvecEtat() {
  // 1. Déclaration de l'état initial (compteur) avec une valeur initiale de 0.
  const [compteur, setCompteur] = useState(0);
  // 2. Utilisation de l'état dans le composant.
  return (
    <div>
      <h1>Composant avec État</h1>
      <p>Compteur : {compteur}</p>
      /* 3. Mise à jour de l'état en réponse à un événement (clic sur le bouton). */
      <button onClick={() => setCompteur(compteur + 1)}>Incrémenter</button>
    </div>
  );
}

export default ComposantAvecEtat;
```

```
jsx

const incremater = () => {
  setCompteur(compteur + 1);
};
```



LES FORMULAIRES

- **Composants d'Entrée Contrôlés** : Pour créer un composant contrôlé, commencez par créer un état local pour stocker la valeur de l'élément de formulaire.

Lier la Valeur à l'État : Utilisez l'état local pour lier la valeur de l'élément de formulaire. Par exemple, pour un champ de saisie de texte :

- L'attribut value est associé à l'état valeur, ce qui rend l'élément de formulaire contrôlé. La fonction onChange est utilisée pour mettre à jour la valeur lorsque l'utilisateur saisit du texte.
- Gestion des Soumissions de Formulaire :
 - Pour gérer la soumission du formulaire, utilisez l'événement onSubmit du formulaire. Vous pouvez empêcher le rechargement de la page en utilisant event.preventDefault().

```
function soumettreFormulaire(event) {
  event.preventDefault();
  // Traitement des données du formulaire ici
}
return (
  <form onSubmit={soumettreFormulaire}>
    <input
      type="text"
      value={valeur}
      onChange={(e) => setValeur(e.target.value)}
    />
    <button type="submit">Envoyer</button>
  </form>
);
```

```
jsx

import React, { useState } from 'react';

function MonFormulaire() {
  const [valeur, setValeur] = useState('');
}
```

```
jsx

<input
  type="text"
  value={valeur}
  onChange={(e) => setValeur(e.target.value)}
/>
```



- Dans cet exemple :
- Nous créons un état local valeur en utilisant le hook useState pour stocker la valeur du champ de saisie.
- Nous utilisons la fonction handleChange pour gérer les modifications de la valeur du champ de saisie en utilisant onChange.
- La fonction soumettreFormulaire est utilisée pour gérer la soumission du formulaire. Nous utilisons event.preventDefault() pour empêcher le rechargement de la page.
- Le champ de saisie de texte est un élément de formulaire contrôlé, lié à l'état valeur. Toute modification dans le champ de saisie met à jour l'état.
- Le bouton de soumission déclenche la fonction soumettreFormulaire lorsqu'il est cliqué.

```
jsx Copy code

import React, { useState } from 'react';

function MonFormulaire() {
  // 1. Création d'un état local pour stocker la valeur du champ de saisie.
  const [valeur, setValeur] = useState('');

  // 2. Fonction pour gérer les changements dans le champ de saisie.
  const handleChange = (e) => {
    setValeur(e.target.value);
  };

  // 3. Fonction pour gérer la soumission du formulaire.
  const soumettreFormulaire = (e) => {
    e.preventDefault(); // Empêche le rechargement de la page.

    // Traitement des données du formulaire.
    alert('Vous avez soumis : ' + valeur);
  };

  return (
    <div>
      <h1>Mon Formulaire React</h1>
      <form onSubmit={soumettreFormulaire}>
        /* 4. Champ de saisie de texte contrôlé */
        <input
          type="text"
          value={valeur}
          onChange={handleChange}
          placeholder="Saisissez quelque chose..."
        />

        /* 5. Bouton de soumission */
        <button type="submit">Soumettre</button>
      </form>
    </div>
  );
}

export default MonFormulaire;
```



LES API

- Les API (Application Programming Interfaces) sont des moyens de communication standardisés permettant à un composant React d'interagir avec des services externes, des bases de données, des services web ou d'autres ressources de données. Dans React, vous pouvez utiliser les API pour récupérer, envoyer, mettre à jour ou supprimer des données, ce qui est essentiel pour le développement d'applications web interactives et dynamiques.
- Voici quelques points clés à retenir sur les API en React :
- **Récupération de Données :**
 - Les API sont couramment utilisées pour récupérer des données à partir de sources externes. Cela peut inclure des données JSON depuis un serveur web, des fichiers, des bases de données, etc.
- **Bibliothèques pour les Requêtes HTTP :**
 - Pour interagir avec une API, vous aurez généralement besoin d'effectuer des requêtes HTTP (GET, POST, PUT, DELETE). Vous pouvez utiliser des bibliothèques comme Axios ou fetch (une fonction JavaScript native) pour gérer ces requêtes.
- **Structure de Réponse :**
 - Les données reçues d'une API sont souvent structurées en JSON (JavaScript Object Notation), qui est un format de données léger et largement utilisé. Vous pouvez extraire les données de la réponse JSON pour les utiliser dans votre application.



- **Sécurité :**
 - Les API peuvent nécessiter des clés d'authentification ou d'autres mesures de sécurité pour accéder aux données. Assurez-vous de comprendre les exigences de sécurité de l'API que vous utilisez.
- **Asynchronicité :**
 - Les appels d'API sont généralement asynchrones, ce qui signifie que votre application ne doit pas être bloquée en attendant la réponse. Vous devez gérer les réponses de manière asynchrone, en utilisant des promesses, des rappels ou des fonctionnalités telles que `async/await`.
- **Actualisation de l'État :**
 - Lorsque vous récupérez des données depuis une API, vous devez souvent mettre à jour l'état de votre composant React pour refléter ces données. Cela peut être accompli en utilisant le hook `useState` ou une gestion d'état plus avancée en fonction des besoins de votre application.
- **Rendu des Données :**
 - Une fois que vous avez récupéré des données depuis une API et mis à jour l'état de votre composant, vous pouvez les afficher dans votre interface utilisateur. Cela implique souvent la cartographie des données dans des éléments JSX pour les rendre visibles à l'utilisateur.



USEEFFECT

- useEffect est un hook en React qui permet de gérer les effets secondaires dans les composants fonctionnels. Les effets secondaires incluent des opérations telles que les requêtes HTTP, la manipulation du DOM, la gestion de l'état, l'abonnement à des flux de données, etc. useEffect est utilisé pour déclencher ces effets secondaires en réaction aux changements d'état ou au montage/démontage du composant.
- Voici une explication détaillée de l'utilisation de useEffect :
- Importation de useEffect :
- Pour utiliser useEffect, vous devez l'importer depuis React.
- Déclaration du Hook useEffect :
- useEffect prend deux arguments : une fonction qui contient le code à exécuter (l'effet secondaire) et un tableau de dépendances. Le tableau de dépendances est une liste d'états ou de valeurs qui détermine quand l'effet doit être exécuté. Si le tableau est vide, l'effet est exécuté une seule fois lors du montage du composant.
- Voici un exemple simple :

jsx

```
import { useEffect } from 'react';
```

jsx

Copy code

```
useEffect(() => {  
  // Code de l'effet à exécuter  
}, []); // Tableau de dépendances vide pour un effet s'exécutant une seule fo
```



- Utilisation de `useEffect` pour effectuer des requêtes HTTP :
- Un cas d'utilisation courant de `useEffect` est de réaliser des requêtes HTTP. Par exemple, vous pouvez utiliser `Axios` pour effectuer une requête `GET` et mettre à jour l'état en fonction de la réponse de l'API.

Gestion des Effets Secondaires :

`useEffect` est utile pour gérer des effets secondaires tels que la mise à jour de l'état, la mise à jour du DOM, l'abonnement à des flux de données, etc. Vous pouvez le personnaliser en fonction des besoins de votre application.

Nettoyage des Effets :

Si un effet a besoin d'un nettoyage lors du démontage du composant, vous pouvez renvoyer une fonction de nettoyage dans la fonction `useEffect`. Cette fonction sera appelée lorsque le composant est démonté (par exemple, lorsque l'utilisateur quitte la page ou navigue ailleurs).

```
jsx Copy code

import { useEffect, useState } from 'react';
import axios from 'axios';

function ComposantAvecEffet() {
  const [donnees, setDonnees] = useState([]);

  useEffect(() => {
    axios.get('https://api.example.com/donnees')
      .then((response) => {
        setDonnees(response.data);
      })
      .catch((erreur) => {
        console.error(erreur);
      });
  }, []); // Tableau de dépendances vide pour un appel API au montage du co
}
```

```
jsx Copy code

useEffect(() => {
  // Code de l'effet à exécuter

  return () => {
    // Code de nettoyage à exécuter lors du démontage du composant
  };
}, []);
```



LE HOOK "USEREF()"

- useRef est un hook en React qui permet de créer des références à des éléments du DOM ou à des valeurs persistantes dans un composant fonctionnel. Cela permet d'accéder directement à ces éléments ou valeurs sans provoquer de nouvelles rendus lorsque leur valeur change. Voici comment utiliser useRef en détail :
- Importation de useRef :
- Pour utiliser useRef, vous devez l'importer depuis React.
- Création d'une Référence :
- Vous pouvez créer une référence en utilisant la fonction useRef(). Par exemple :
- initialValue est la valeur initiale que vous souhaitez attribuer à la référence. Il peut être nul (null) si vous ne voulez pas d'initialisation.
- Accès aux Éléments du DOM :
- Vous pouvez utiliser useRef pour accéder aux éléments du DOM directement. Par exemple, si vous avez un élément avec un identifiant monElement :

jsx

```
import { useRef } from 'react';
```

jsx

```
const monRef = useRef(initialValue);
```

jsx

```
const monElementRef = useRef(null);

useEffect(() => {
  // Accès direct à l'élément du DOM
  monElementRef.current.focus();
}, []);
```



- Stockage de Valeurs Persistantes :
- `useRef` peut également être utilisé pour stocker des valeurs persistantes entre les rendus sans provoquer de réactualisation du composant. Par exemple, vous pouvez stocker un compteur :
- La valeur du compteur est stockée dans `compteurRef.current`.
- Utilisation d'une Référence dans le JSX :
- Pour utiliser une référence dans le JSX, vous pouvez l'assigner à la propriété `ref` d'un élément.

```
jsx
<input ref={monElementRef} />
```

```
jsx
const compteurRef = useRef(0);

// ...

// Incrémentation du compteur sans déclencher de rendu
compteurRef.current++;
```

Cela peut être utile pour accéder à l'élément du DOM, pour la gestion de formulaires ou pour d'autres interactions avec le DOM.

Accès à la Valeur Courante :

Pour accéder à la valeur courante d'une référence, utilisez `ref.current`.

Cela vous permet d'accéder à la valeur ou à l'élément du DOM stocké dans la référence.

```
jsx
const valeurCourante = monRef.current;
```



- Nous importons `useRef` et `useEffect` de `React`.
- Nous créons une référence `monElementRef` à l'aide de `useRef(null)`. La valeur initiale est `null`.
- Dans la fonction `useEffect`, nous utilisons `monElementRef.current.focus()` pour mettre automatiquement le focus sur l'élément d'entrée lorsque le composant est monté. Le tableau de dépendances vide `[]` signifie que cet effet ne sera exécuté qu'au moment du montage.
- Dans le JSX, nous assignons `monElementRef` à la propriété `ref` de l'élément d'entrée. Cela nous permet d'accéder à cet élément du DOM directement via `monElementRef.current`.
- Lorsque le composant est monté, l'élément d'entrée reçoit automatiquement le focus, ce qui signifie que le curseur clignotant est positionné à l'intérieur de l'élément d'entrée pour que l'utilisateur puisse commencer à saisir du texte sans avoir à cliquer sur l'élément.

```
jsx
Copy code

import React, { useRef, useEffect } from 'react';

function ComposantAvecFocusAutomatique() {
  // Créez une référence pour l'élément d'entrée
  const inputElementRef = useRef(null);

  // Utilisez 'useEffect' pour mettre automatiquement le focus sur l'élément
  useEffect(() => {
    inputElementRef.current.focus();
  }, []);

  return (
    <div>
      <h1>Exemple d'utilisation de useRef pour le focus automatique</h1>
      <input
        type="text"
        ref={inputElementRef}
        placeholder="Saisissez quelque chose ici"
      />
      <p>Commencez à saisir pour voir l'effet de focus automatique.</p>
    </div>
  );
}

export default ComposantAvecFocusAutomatique;
```



UTILISATION DE "USEMEMO" ET "REACT.MEMO"

- `useMemo` et `React.memo` sont deux outils qui peuvent améliorer les performances de votre application React en évitant les calculs inutiles et les ré-renderings de composants. Commençons par expliquer ces deux concepts, puis nous verrons comment les utiliser.
- `useMemo` :
 - `useMemo` est un hook en React qui permet de mémoriser le résultat d'une fonction de calcul, afin que cette fonction ne soit pas réexécutée à chaque rendu du composant. Cela est utile lorsque vous avez une fonction de calcul coûteuse en termes de performances, mais dont le résultat dépend des valeurs présentes dans les dépendances.
 - `fonctionDeCalcul` est la fonction que vous souhaitez optimiser.
 - `parametres` est un tableau contenant les dépendances sur lesquelles le calcul dépend.
 - `resultat` contient le résultat mémorisé du calcul.
- `React.memo` :
 - `React.memo` est une fonction qui peut être utilisée pour envelopper un composant fonctionnel. Elle permet de mémoriser le rendu du composant afin qu'il ne soit pas réexécuté inutilement. Cela est utile lorsque les propriétés (props) d'un composant n'ont pas changé et que le composant n'a pas besoin d'être re-rendered.

```
jsx  
Copy code  
const resultat = useMemo(() => fonctionDeCalcul(parametres), [parametres]);
```

```
jsx  
const ComposantMemo = React.memo((props) => {  
  // Render le composant ici  
});
```



- Maintenant, voyons comment utiliser ces deux concepts ensemble pour améliorer les performances d'un composant. Supposons que vous avez un composant `ListeElements` qui affiche une liste d'éléments à partir de ses propriétés (props).
- Si la liste `elements` n'est pas censée changer fréquemment, vous pouvez envelopper le composant avec `React.memo` pour éviter les ré-renderings inutiles.
- De plus, si vous avez une fonction de calcul coûteuse qui transforme les `elements` en une autre forme (par exemple, en les triant), vous pouvez utiliser `useMemo` pour mémoriser le résultat de cette transformation.
- Ainsi, le tri de `elements` n'aura lieu que si `elements` change, évitant des calculs inutiles lors des ré-renderings du composant.

```
jsx
function ListeElements({ elements }) {
  return (
    <ul>
      {elements.map((element) => (
        <li key={element.id}>{element.nom}</li>
      ))}
    </ul>
  );
}
```

```
jsx
const ListeElementsMemo = React.memo(ListeElements);
```

- **useMemo :**
 - Type : Il s'agit d'un hook, c'est-à-dire une fonction qui est utilisée dans un composant fonctionnel.
 - Objectif : `useMemo` est principalement utilisé pour mémoriser le résultat d'une fonction de calcul complexe. Il permet d'optimiser les calculs dans le composant en évitant de recalculer la valeur si les dépendances n'ont pas changé. Cela peut aider à améliorer les performances en évitant des calculs coûteux.
 - Utilisation : Vous enveloppez une fonction de calcul dans `useMemo` en spécifiant les dépendances. Le résultat de la fonction sera mémorisé et renvoyé si les dépendances restent inchangées.
- **React.memo :**
 - Type : Il s'agit d'une fonction de niveau supérieur (HOC, Higher Order Component) qui enveloppe un composant fonctionnel.
 - Objectif : `React.memo` est principalement utilisé pour mémoriser le rendu d'un composant entier. Il permet d'optimiser les performances en évitant de réexécuter le rendu du composant si les propriétés (props) n'ont pas changé. Cela est utile pour éviter des ré-renderings inutiles.
 - Utilisation : Vous enveloppez un composant fonctionnel avec `React.memo` en passant le composant comme argument. Le composant résultant est mémorisé et ne sera réexécuté que si les propriétés changent.

```
jsx
const ListeElements = ({ elements }) => {
  const elementsTries = useMemo(() => {
    return [...elements].sort((a, b) => a.nom.localeCompare(b.nom));
  }, [elements]);

  return (
    <ul>
      {elementsTries.map((element) => (
        <li key={element.id}>{element.nom}</li>
      ))}
    </ul>
  );
};
```



- Exemple useMemo :
- Supposons que vous ayez un composant qui affiche une liste de nombres générés aléatoirement. Vous souhaitez trier cette liste à l'aide d'une fonction de tri coûteuse, mais vous ne voulez pas que le tri soit effectué à chaque rendu du composant, car la liste n'est censée changer que rarement.
- Voici comment vous pouvez utiliser useMemo pour optimiser le tri de la liste :
- Explication :
 1. Nous utilisons useState pour suivre l'ordre de tri (croissant ou décroissant) de la liste.
 2. Nous utilisons useMemo pour trier la liste générée aléatoirement, stockée dans listeAleatoire. La fonction passée à useMemo effectue le tri de la liste en fonction de l'ordre de tri actuel (triCroissant).
 3. La dépendance de useMemo est un tableau contenant les valeurs qui, lorsqu'elles changent, déclenchent le recalcul de listeTrie. Dans ce cas, ce sont listeAleatoire et triCroissant.
 4. Lorsque l'utilisateur clique sur le bouton pour changer l'ordre de tri, triCroissant change, ce qui déclenche le recalcul de listeTrie. Le tri est effectué en fonction du nouvel ordre de tri.

```
1 import React, { useState, useMemo } from 'react';
2
3 function ListeTrie() {
4   // Générez une liste de nombres aléatoires
5   const listeAleatoire = Array.from({ length: 1000 }, () =>
6     Math.floor(Math.random() * 1000)
7   );
8
9   // Utilisez `useState` pour suivre l'ordre de tri
10  const [triCroissant, setTriCroissant] = useState(true);
11
12  // Utilisez `useMemo` pour trier la liste uniquement lorsque `li
13  steAleatoire` ou `triCroissant` change
14  const listeTrie = useMemo(() => {
15    return triCroissant
16      ? [...listeAleatoire].sort((a, b) => a - b)
17      : [...listeAleatoire].sort((a, b) => b - a);
18  }, [listeAleatoire, triCroissant]);
19
20  return (
21    <div>
22      <h1>Liste Triée</h1>
23      <button onClick={() => setTriCroissant(!triCroissant)}>
24        Changer l'ordre de tri
25      </button>
26      <ul>
27        {listeTrie.map((nombre, index) => (
28          <li key={index}>{nombre}</li>
29        ))}
30      </ul>
31    </div>
32  );
33 }
34 export default ListeTrie;
35
```



- Exemple react,memo
- Supposons que vous ayez un composant qui affiche une liste de noms d'utilisateurs. Vous souhaitez éviter des ré-renderings inutiles de ce composant lorsque la liste de noms reste la même. Pour ce faire, vous pouvez utiliser React.memo pour mémoriser le rendu du composant.
- Voici comment vous pouvez utiliser React.memo :
- Explication :
 - Nous avons un composant fonctionnel ListeNoms qui prend une liste de noms (noms) en tant que propriété (props) et affiche cette liste. Dans cet exemple, nous utilisons console.log pour indiquer quand le composant est rendu.
 - Nous utilisons React.memo pour envelopper ListeNoms et créons un nouveau composant appelé ListeNomsMemo. Cela signifie que ListeNomsMemo sera mémorisé, c'est-à-dire qu'il ne sera réexécuté que si les propriétés (props) changent.
 - À chaque rendu de ListeNoms, la console affiche "Rendu de ListeNoms". Cependant, lorsque les propriétés restent les mêmes, le composant n'est pas réexécuté, ce qui signifie que "Rendu de ListeNoms" ne s'affiche pas à nouveau.
 - Cela améliore les performances en évitant des ré-renderings inutiles du composant lorsque la liste de noms reste la même.

```
1 import React from 'react';
2
3 // Composant fonctionnel qui affiche une liste de noms
4 const ListeNoms = ({ noms }) => {
5   console.log('Rendu de ListeNoms');
6   return (
7     <div>
8       <h1>Liste de Noms</h1>
9       <ul>
10         {noms.map((nom, index) => (
11           <li key={index}>{nom}</li>
12         ))}
13       </ul>
14     </div>
15   );
16 };
17
18 // Enveloppez le composant dans `React.memo` pour le mémoriser
19 const ListeNomsMemo = React.memo(ListeNoms);
20
21 export default ListeNomsMemo;
```



LE HOOK USECALLBACK.

- `useCallback` est un hook en React qui permet de mémoriser une fonction (callback) afin d'éviter sa recreation à chaque rendu du composant. Cela est particulièrement utile lorsque vous devez transmettre des fonctions en tant que propriétés (props) à des composants enfants ou lorsque vous utilisez des fonctions comme dépendances dans d'autres hooks.
- Voici comment utiliser `useCallback` en détail :
- Importation de `useCallback` :
- Pour utiliser `useCallback`, vous devez l'importer depuis React.
- Création d'une Fonction à Mémoriser :
- Vous pouvez créer une fonction que vous souhaitez mémoriser à l'aide de `useCallback`. Par exemple :
- `maFonctionMemo` contient la fonction mémorisée.
- `dependances` est un tableau des valeurs sur lesquelles la fonction dépend. La fonction ne sera recalculée que si les valeurs dans ce tableau changent.
- Utilisation de la Fonction Mémorisée :
- Vous pouvez utiliser `maFonctionMemo` comme n'importe quelle autre fonction JavaScript.
- Utilisation de `useCallback` pour Optimiser le Rendu :
- L'utilisation la plus courante de `useCallback` est pour optimiser le rendu des composants en évitant qu'ils ne soient créés à chaque rendu. Par exemple, si vous transmettez des fonctions en tant que propriétés (props) à des composants enfants, vous pouvez les mémoriser pour éviter des ré-renderings inutiles de ces composants.
- Dans cet exemple, `handleClick` est mémorisé et ne change pas entre les rendus de `ComposantParent`, ce qui améliore les performances.

```
jsx
import { useCallback } from 'react';
```

```
jsx
const maFonctionMemo = useCallback(() => {
  // Code de la fonction
}, [dependances]);
```

```
jsx
return (
  <div>
    <button onClick={maFonctionMemo}>Cliquez ici</button>
  </div>
);
```

```
jsx
const ComposantParent = () => {
  const handleClick = useCallback(() => {
    // Traitement à effectuer lors du clic
  }, []);

  return <ComposantEnfant onClick={handleClick} />;
};
```



- Exemple
- Supposons que vous ayez un composant qui affiche une liste de tâches à accomplir. Vous souhaitez permettre à l'utilisateur de cocher une tâche comme terminée en cliquant sur un bouton. Vous voulez également compter combien de tâches sont terminées. Cependant, vous ne voulez pas que chaque clic sur un bouton crée une nouvelle fonction `toggleTask` inutile.
- Voici comment vous pouvez utiliser `useCallback` pour mémoriser la fonction `toggleTask` :
- Explication :
 1. Nous avons un composant `ListeDeTaches` qui gère un état `taches` sous forme de tableau d'objets, chaque objet représentant une tâche avec un identifiant, un texte et un état de terminaison.
 2. Nous utilisons `useCallback` pour mémoriser la fonction `toggleTask` qui permet de basculer l'état de terminaison d'une tâche. Cette fonction est mémorisée avec un tableau de dépendances vide (`[]`) car elle ne dépend d'aucune valeur externe. Cela signifie que `toggleTask` ne sera pas recréée à chaque rendu.
 3. Lorsque l'utilisateur clique sur la case à cocher d'une tâche, la fonction `toggleTask` est appelée, ce qui met à jour l'état des tâches.
 4. Nous utilisons `filter` pour compter le nombre de tâches terminées. Comme `toggleTask` est mémorisée, cela n'entraîne pas de ré-rendering inutile du composant.

```
1 import React, { useState, useCallback } from 'react';
2
3 function ListeDeTaches() {
4   const [taches, setTaches] = useState([
5     { id: 1, texte: 'Faire les courses', terminee: false },
6     { id: 2, texte: 'Préparer le dîner', terminee: false },
7     // ... d'autres tâches ...
8   ]);
9
10  // Utilisez `useCallback` pour mémoriser la fonction `toggleTask`
11  const toggleTask = useCallback((id) => {
12    setTaches((tachesActuelles) =>
13      tachesActuelles.map((tache) =>
14        tache.id === id ? { ...tache, terminee: !tache.terminee } : tache
15      )
16    );
17  }, []);
18
19  // Utilisez `filter` pour compter les tâches terminées
20  const tachesTerminees = taches.filter((tache) => tache.terminee);
21
22  return (
23    <div>
24      <h1>Liste de Tâches</h1>
25      <ul>
26        {taches.map((tache) => (
27          <li key={tache.id}>
28            <input
29              type="checkbox"
30              checked={tache.terminee}
31              onChange={() => toggleTask(tache.id)}
32            />
33            {tache.texte}
34          </li>
35        ))}
36      </ul>
37      <p>{tachesTerminees.length} tâche(s) terminée(s).</p>
38    </div>
39  );
40 }
41
42 export default ListeDeTaches;
43
```

HOOK PERSONNALISÉ

- Un Hook personnalisé est une fonction JavaScript que vous créez pour encapsuler de la logique réutilisable dans une application React. Vous pouvez créer des Hooks personnalisés pour regrouper des fonctionnalités spécifiques et les partager entre différents composants.
- Voici comment créer un Hook personnalisé en React :
- Créez un fichier pour votre Hook :
- Commencez par créer un fichier (par exemple, useCustomHook.js) pour votre Hook personnalisé.
- Importez useState, useEffect, ou d'autres Hooks si nécessaire :
- Selon les fonctionnalités que vous souhaitez encapsuler dans votre Hook, importez les Hooks de React nécessaires.
- Définissez votre Hook personnalisé :
- Dans le fichier de votre Hook personnalisé, créez votre fonction qui encapsule la logique que vous souhaitez partager. Voici un exemple :
- Dans cet exemple, useCustomHook encapsule la gestion d'un état (valeur) et un effet pour mettre à jour le titre du document en fonction de cette valeur. Il expose également une fonction increment pour incrémenter la valeur.

```
jsx
Copy code

import { useState, useEffect } from 'react';

function useCustomHook(initialValue) {
  const [valeur, setValeur] = useState(initialValue);

  useEffect(() => {
    // Logique d'effet (par exemple, une mise à jour du document.title)
    document.title = `Valeur : ${valeur}`;
  }, [valeur]);

  function increment() {
    setValeur(valeur + 1);
  }

  return { valeur, increment };
}

export default useCustomHook;
```



- Utilisez votre Hook personnalisé dans un composant :
- Pour utiliser votre Hook personnalisé dans un composant, importez-le et appelez-le. Voici comment vous pouvez l'utiliser :
- Dans cet exemple, useCustomHook est importé et utilisé dans le composant ComposantUtilisantCustomHook. Le composant bénéficie de la logique encapsulée dans le Hook personnalisé.

```
jsx

import React from 'react';
import useCustomHook from './useCustomHook';

function ComposantUtilisantCustomHook() {
  const { valeur, increment } = useCustomHook(0);

  return (
    <div>
      <h1>Composant Utilisant le Hook Personnalisé</h1>
      <p>Valeur : {valeur}</p>
      <button onClick={increment}>Incrémenter</button>
    </div>
  );
}

export default ComposantUtilisantCustomHook;
```



REACT ROUTER

- React Router est une bibliothèque JavaScript populaire conçue pour gérer la navigation dans les applications React. Elle permet de créer des applications à navigation fluide en dirigeant les utilisateurs vers différentes vues ou pages en fonction des URL

- Installation

```
bash
npm install react-router-dom
```

- BrowserRouter est un composant clé de React Router qui doit être utilisé pour englober toute l'application et permettre la gestion de la navigation basée sur les URL. L'utilisation de BrowserRouter se fait généralement dans le fichier index.js de votre application React.

```
import { BrowserRouter } from "react-router-dom";
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```



- Vous devez toujours importer les composants nécessaires depuis React Router :
 - Configuration des Routes : Utilisez les composants Routes et Route pour configurer vos routes. Voici un exemple simple de configuration de routes :
- Utilisez Routes en tant que composant racine pour définir un ensemble de routes.
 - À l'intérieur de Routes, chaque route est définie avec le composant Route. Le path spécifie l'URL associée à la route, et l'élément element spécifie le composant à afficher lorsque l'URL correspond.
 - Affichage des Composants : Les composants correspondants seront rendus lorsque l'URL correspond à un path spécifié. Dans cet exemple, les composants Accueil et APropos seront rendus respectivement lorsque l'utilisateur visite l'URL d'accueil (/) et l'URL "À Propos" (/a-propos).

jsx

```
import { Link, Routes, Route } from 'react-router-dom';
```

jsx

```
function App() {
  return (
    <div>
      <nav>
        <ul>
          <li>
            <Link to="/">Accueil</Link>
          </li>
          <li>
            <Link to="/a-propos">À Propos</Link>
          </li>
        </ul>
      </nav>

      <Routes>
        <Route path="/" element={<Accueil />} />
        <Route path="/a-propos" element={<APropos />} />
      </Routes>
    </div>
  );
}
```



- Pour utiliser des paramètres d'URL dans React Router v6, vous pouvez capturer des parties dynamiques de l'URL et les transmettre à vos composants. Voici comment cela fonctionne :

- Définir des Routes avec des Paramètres : Vous pouvez définir des routes avec des paramètres en utilisant des `:` dans le path. Par exemple :

```
jsx
<Route path="/utilisateur/:id" element={<Utilisateur />} />
```

- Dans cet exemple, `:id` est un paramètre d'URL dynamique qui peut correspondre à n'importe quelle valeur dans l'URL, par exemple, `"/utilisateur/123"`.
- Accéder aux Paramètres : Pour accéder aux paramètres dans votre composant, vous pouvez utiliser le hook `useParams` fourni par React Router. Voici comment vous pouvez l'utiliser :

- Dans cet exemple, `useParams` vous permet d'extraire la valeur de l'ID à partir de l'URL. Vous pouvez ensuite utiliser cette valeur comme bon vous semble dans votre composant.
- Navigation avec des Paramètres : Vous pouvez également générer des liens avec des paramètres d'URL en utilisant `Link` et en passant les paramètres dans l'URL. Par exemple

```
jsx
<Link to="/utilisateur/123">Voir le profil</Link>
```

```
jsx
import { useParams } from 'react-router-dom';

function Utilisateur() {
  const { id } = useParams();

  return <div>Profil de l'utilisateur avec l'ID {id}</div>;
}
```

En JavaScript, `const { id }` est une syntaxe utilisée pour extraire une propriété d'un objet en utilisant la déstructuration (destructuring) d'objets. Cette syntaxe permet d'assigner la valeur de la propriété `id` de l'objet à une variable portant le même nom, dans ce cas, `id`.

Voici un exemple pour illustrer son fonctionnement :

```
javascript
const utilisateur = { id: 123, nom: 'John', age: 30 };

const { id } = utilisateur;

console.log(id); // Affiche : 123
```



